

Requisitos funcionais e não funcionais

Versão 3 — 22/09/2026. Lista o que o copiloto de verificação precisa fazer e como precisa se comportar. Cada requisito aqui está amarrado a uma spec do OpenSpec, a uma medição registrada no repositório ou a uma decisão do grupo — nenhum é preferência sem lastro.

Esta página é para leitura humana, então está escrita em linguagem simples. As palavras-chave em inglês (SHALL, MUST NOT) continuam existindo, mas só dentro dos arquivos de spec em `openspec/`, onde são exigidas pelo validador. Aqui, "precisa" tem o mesmo peso que SHALL e "não pode" tem o mesmo peso que MUST NOT.

Cada requisito traz por que ele existe e em que pé está:

Estado	O que significa
especificado	já existe spec escrita que cobre o requisito; falta implementar
medido	existe medição no repositório que sustenta ou restringe o requisito
em aberto	o grupo concorda que é requisito, mas ainda não há spec nem medição
adiado	está fora do MVP; para entrar, precisa de um change próprio

Se esta página divergir de uma spec, a spec vence. A correção é atualizar a spec ou o requisito, nunca ajustar o texto ao que já foi feito.

Requisitos funcionais

Requisito funcional é o que o sistema faz — o comportamento que o usuário consegue observar.

RF01 — Usar sem criar conta · *especificado*

O usuário cola ou encaminha um texto, ou envia um link, e recebe a resposta. O sistema não pode pedir cadastro, login nem preenchimento de formulário antes da primeira verificação.

Por quê: o público-alvo tem baixo letramento textual e recebe a mensagem suspeita pelo WhatsApp. Qualquer tela de cadastro entre a dúvida e a resposta é motivo de abandono. Vem da spec `acessibilidade-leitura`.

RF02 — Achar a alegação antes de procurar evidência · *especificado*

Antes de buscar qualquer coisa, o sistema identifica qual é a afirmação de saúde dentro do texto e mostra ao usuário qual alegação está sendo verificada. Se houver mais de uma, verifica a de maior risco e oferece as outras. Se o texto for opinião ou desabafo, explica a diferença entre opinião e afirmação verificável, e não dá veredito.

Por quê: mensagem encaminhada costuma misturar várias coisas. Sem esse passo, o sistema busca pelo texto inteiro e recupera evidência sobre o assunto errado. Vem da spec `verificacao-alegacao`.

RF03 — Buscar em dois índices, português primeiro · *medido*

A busca combina um índice que compara palavras (BM25) com um índice que compara significado (embeddings). As fontes em português são esgotadas antes de recorrer a fontes em inglês.

Por quê: medimos as três configurações. Só palavras acerta 85% das consultas nos 5 primeiros resultados; a busca por significado e a combinação das duas acertam 100%. A combinação é a melhor em ordenação. A regulagem aferida é peso 0,9 para o lado semântico, com o modelo `e5-base` — e ela não se transfere para outro modelo de embedding. Ver Aferição da recuperação.

RF04 — Responder só pelo que foi recuperado · *medido*

O sistema responde apenas com base no trecho de evidência que a busca trouxe. Não pode dar veredito usando o que o modelo "sabe" de fábrica. Se a resposta produzir uma frase que nenhum trecho sustenta, essa frase é retirada.

Por quê: é o que separa este projeto de um chute bem escrito. Na sonda T1 o modelo recebeu uma pergunta sobre a vacina Qdenga sem trecho que a cobrisse, e respondeu "evidência insuficiente" em vez de inventar — o conhecimento estava no texto entregue, não no modelo. Vem da spec `verificacao-alegacao`.

RF05 — Quatro respostas possíveis, e uma delas é "não sei" · *especificado*

O veredito é sempre um entre quatro: falso, verdadeiro, verdadeiro fora de contexto ou exagerado, ou evidência insuficiente. Uma alegação que parece absurda mas que a evidência sustenta recebe verdadeiro — o sistema não rebaixa o veredito só porque a afirmação soa estranha.

Por quê: a maior parte da desinformação em saúde não é mentira inteira, é verdade fora de contexto. Um sistema com só dois rótulos força essas mensagens para um dos extremos. Vem da spec `verificacao-alegacao`.

RF06 — "Não temos" é diferente de "não existe" · *especificado*

Quando o assunto é mais recente que o acervo, o sistema diz que a lacuna é do acervo, informa a data de corte e, se o índice de localização encontrar uma checagem publicada, mostra agência, data e endereço. Nunca apresenta a falta de material como se ninguém no mundo tivesse checado aquilo.

Por quê: o corpus com texto integral termina em maio de 2021. Buscamos e confirmamos: `qdenga`, `mpox`, `oropouche` e `semaglutida` têm zero ocorrência. Se o sistema disser só "não encontrei", o usuário entende que a notícia não foi desmentida. Vem das specs `frescor-corpus` e `fix-resposta-sem-evidencia`.

RF07 — Pedido de conduta médica sai do caminho normal · *medido*

Antes da verificação, um classificador separado detecta se a pessoa está pedindo conduta clínica para o caso dela: começar, parar ou trocar tratamento, dose, diagnóstico, leitura de exame. Nesses casos a mensagem é desviada para redirecionamento a profissional de saúde. Sinal de urgência interrompe tudo e orienta procurar atendimento.

Por quê: na sonda T2 perguntamos se a mãe com câncer podia parar a quimioterapia e tomar um chá. O modelo abriu a resposta com "VEREDITO: Falso" e só citou procurar médico no último bloco. Quem lê rápido entende o "Falso" como resposta à pergunta que fez. A conclusão foi que instrução no prompt não segura essa fronteira: precisa de classificador próprio, rodando antes. Vem da spec `fronteira-orientacao-saude`.

RF08 — Desmentir sem receitar · *especificado*

Ao derrubar uma alegação sobre tratamento, o sistema explica por que ela não se sustenta e para por aí. Não pode sugerir outro tratamento, substância ou conduta no lugar.

Por quê: indicar substituto é prescrever, e o sistema não é profissional de saúde. Vem da spec `fronteira-orientacao-saude`.

RF09 — Quatro blocos, em dois formatos · *especificado*

Toda resposta tem quatro blocos, mas o formato muda conforme haja evidência ou não.

Com evidência: (1) o veredito, (2) o que se sabe sobre o assunto, (3) por que aquela mensagem engana, (4) o que observar da próxima vez.

Sem evidência: (1) o veredito e o que foi procurado, (2) por que não achar não é desmentir, (3) o que a pessoa pode conferir por conta própria, (4) onde procurar.

O sistema não pode usar um formato no lugar do outro.

Por quê: a primeira redação da spec exigia os mesmos quatro blocos sempre. Quando o veredito era "evidência insuficiente", o bloco "por que engana" pedia ao sistema que afirmasse sobre a mensagem exatamente aquilo que o veredito acabara de declarar desconhecido — e na sonda o modelo degenerou, repetindo "evidência insuficiente" dentro do bloco. Corrigido pelo change `fix-resposta-sem-evidencia`.

RF10 — Catálogo fechado de técnicas de manipulação · especificado

No bloco "por que engana", o sistema nomeia pelo menos uma técnica de uma lista fixa de 6 a 8 rótulos, guardada em arquivo versionado, escrita em linguagem do dia a dia. Exemplos do catálogo: `cura milagrosa`, `manchete exagerada`. Na resposta sem evidência essa exigência não vale — nomear técnica ali é defeito, e o rótulo é removido, não trocado por outro.

Por quê: lista fechada é o que permite testar a saída automaticamente e o que impede o modelo de inventar categorias acadêmicas que o usuário não entende.

RF11 — Escolher o rótulo certo, não qualquer rótulo do catálogo · em aberto

Quando mais de um rótulo do catálogo cabe, o sistema escolhe o mais adequado à alegação. A validação automática precisa distinguir duas coisas diferentes: se o rótulo **pertence** ao catálogo e se ele **serve** àquele caso.

Por quê: na sonda, a mesma alegação de cura do câncer recebeu `cura milagrosa` numa execução e `manchete exagerada` noutra. Os dois estão no catálogo, então a validação prevista aprovaria ambos — mas para uma promessa de cura, `manchete exagerada` é o rótulo errado. Ainda não há spec nem task: é decisão de grupo pendente.

RF12 — Fonte sempre citada, detalhe em segunda camada · especificado

O sistema nomeia a agência que fez a checagem e leva ao endereço original. Não pode reproduzir a checagem inteira — usa citação curta com paráfrase. Fontes, trechos citados, datas e detalhe de método ficam numa camada que o usuário abre quando quer, sem bagunçar a resposta principal.

Por quê: duas exigências ao mesmo tempo. O usuário precisa poder conferir e discordar, e a camada visível precisa caber em 120 palavras. Vem das specs `recuperacao-evidencia` e `resposta-formativa`.

RF13 — Separar fato, evidência e opinião · especificado

No bloco 2, o sistema mostra qual parte da mensagem é afirmação verificável, qual é a evidência apresentada e com que força, e qual é opinião. Essa separação é independente do veredito: ter opinião no texto não torna a mensagem falsa, e evidência fraca não vira veredito falso.

Por quê: é uma das três competências que o enunciado do challenge exige, e é o caso mais comum de mensagem que parte de um fato verdadeiro para concluir algo que o fato não sustenta. Vem da spec `verificacao-alegacao`.

RF14 — Padronizar o vocabulário de veredito antes de indexar · medido

Os rótulos que cada agência usa são convertidos para um vocabulário único, por um mapa versionado, antes de qualquer indexação.

Por quê: cada agência escreve o veredito do seu jeito. Medido no FactCenter: **26 grafias distintas de veredito** — e não 285, que é a contagem de strings serializadas do campo `rating` e mede outra coisa. O mapa está em `tratamento/mapa_vereditos.json`, versão 1.0.0, 19 chaves. Ver Tratamento dos datasets.

RF15 — Conjunto de casos para o teste com usuário · especificado

O projeto mantém de 20 a 30 casos curados, com pelo menos quatro itens verdadeiros contra-intuitivos e de 4 a 6 itens-armadilha, em que a saída da IA apresenta evidência ambígua ou dedução errada de propósito. Itens verdadeiros não podem vir de agências de checagem.

Por quê: os itens-armadilha são o único jeito de medir aceitação cega — se a pessoa concorda com a ferramenta mesmo quando ela erra. E itens verdadeiros tirados de agência de checagem enviam o teste, porque agência publica principalmente desmentido. Vem da spec [avaliacao-instrumento](#).

Requisitos adiados

Os dois requisitos abaixo foram levantados pelo grupo e não estão descartados. Estão fora do MVP até existir um change que os cubra e resolva o consentimento.

RF-A01 — Cadastro de cliente

Conta com e-mail e senha. Conflita com RF01 enquanto for condição para usar. Só é admissível como conta **opcional**, oferecida depois da primeira verificação.

RF-A02 — Login e histórico de pesquisa

Mesma restrição do RF-A01, mais o peso da LGPD descrito no RNF11: guardar histórico de pesquisa de saúde é guardar dado pessoal sensível.

Requisitos não funcionais

Requisito não funcional é como o sistema precisa se comportar — velocidade, segurança, legibilidade, limites de uso dos dados.

RNF01 — Texto curto e sem jargão · *especificado*

A camada visível tem no máximo **120 palavras**, com frases de até **20 palavras**. Termo técnico só aparece com a tradução ao lado. Jargão de método — "revisão por pares", "evidência preliminar" — não entra sem explicação em linguagem cotidiana.

Por quê: o público é de baixo letramento textual. Vem da spec [acessibilidade-leitura](#).

RNF02 — Interface com números conferíveis · *especificado*

Contraste mínimo de 4,5:1 no texto, fonte base de pelo menos 18 px que acompanha o ajuste de tamanho do sistema, área de toque de pelo menos 44 px e uma única ação principal por tela.

Por quê: "interface intuitiva" não é testável. Estes quatro números são. Vem da spec [acessibilidade-leitura](#).

RNF03 — Funciona no celular e no computador · *especificado*

A interface se adapta a navegador móvel e de desktop. Com a fonte do sistema ampliada, o texto não corta, não sobrepõe e não força rolagem lateral.

Por quê: a mensagem suspeita chega pelo celular, e o usuário com dificuldade de leitura costuma estar com a fonte ampliada.

RNF04 — Resposta em streaming, porque a geração demora · *medido*

A busca responde em menos de 1 segundo. A escrita da resposta leva de 17 a 23 segundos e é ela que domina o tempo total — por isso o texto aparece aos poucos, em vez de esperar a resposta ficar pronta.

Por quê: o alvo inicial do grupo era de 3 segundos no total. É inatingível com o gerador escolhido. Medimos: 38 ms de mediana na busca combinada, 13 ms na busca por significado — dentro do orçamento, a busca é ruído. Vinte segundos de tela parada, porém, é abandono provável no teste com usuário. Ver Sonda do gerador.

RNF05 — O gerador roda sempre com o raciocínio desligado · *medido*

Toda chamada ao modelo passa `think: false`.

Por quê: com o raciocínio ligado, o tempo sobe para 112 segundos e só 1 das 3 sondas produziu resposta — as outras duas gastaram todo o orçamento de tokens pensando e devolveram texto vazio. Houve um ganho isolado de qualidade de rótulo, que não compensa duas respostas perdidas em três.

RNF06 — Tudo rastreável até a checagem original · *especificado*

Qualquer afirmação do sistema precisa poder ser seguida até a checagem de onde veio. O veredito sempre aparece junto do critério que o sustenta; veredito sozinho, sem raciocínio nem procedência, é proibido.

Por quê: é o princípio que sustenta o projeto inteiro. A pergunta do challenge é como a IA ajuda a avaliar confiabilidade **sem substituir o pensamento crítico** — e veredito nu é exatamente o que substitui. Está em `openspec/project.md`.

RNF07 — Trecho citado idêntico ao original · *medido*

Todo fragmento exibido é conferido contra o texto de origem por igualdade literal — sem dobrar acento, sem colapsar espaço. Base que não passa nesse portão não fornece trecho para exibição.

Duas bases não passam, por motivos opostos:

- **FACTCK.BR** perdeu caracteres. Onze maiúsculas acentuadas têm zero ocorrência (Ã Ä Å Ê Ë Ì Í Î Ï Ñ Ò Ó Ô Õ Ö Ù Ü Ç), sendo que sete delas têm minúscula frequente — ã contra 3.625 ã, ç contra 2.312 ç. A perda já está no arquivo distribuído pelos autores, então **não tem conserto**: nenhuma reexecução recupera caractere que não existe na fonte. A base entra por contagem de rótulo, não por citação.
- **FakeRecogna 2.0** entrega o texto lematizado e sem maiúsculas na origem — `o governar equador anunciar preparar cova`. O efeito é o mesmo, a acusação não: não é corrupção, é transformação declarada pelos autores.

Medido sobre 500 registros: 4.968 fragmentos, zero infielis. Ver Tratamento dos datasets.

RNF08 — Nenhum classificador de veredito treinado em rótulo binário · *especificado*

FakeRecogna, PUBHEALTH e Med-MMHL servem como banco de estímulos para o teste com usuário, nunca como alvo de treino de um classificador de verdadeiro/falso. Treino continua permitido num lugar só: classificadores auxiliares pequenos que decidem **comportamento**, como o de RF07.

Por quê: um classificador treinado em rótulos devolveria "falso" sem conseguir mostrar de onde tirou aquilo, o que derruba o RNF06. Regra registrada em `openspec/project.md`.

RNF09 — Texto em inglês não chega ao usuário · *especificado*

PUBHEALTH, FakeHealth e InSciOut são instrumentos: definem critérios e formato interno do prompt, e o texto deles não aparece na tela.

Evidência recuperada em inglês é outro caso: aparece parafraseada em português na camada visível, com o trecho original disponível na camada de detalhe.

Por quê: as duas situações são fáceis de confundir, e a distinção muda o que aparece na tela. Dataset instrumental nunca vira conteúdo; fonte em inglês pode virar, desde que traduzida e auditável.

RNF10 — Licença respeitada por base · *medido*

- **FakeRecogna 2.0: MIT**, declarada pelos autores. A pendência de licença que o inventário registrava era da versão 1 da base, substituída em 19/09/2026.
- **Telegram antivacina: CC BY-NC 4.0**. A cláusula de uso não comercial **propaga para todo derivado**, e está no cabeçalho de cada arquivo gerado. Nenhum texto de post e nenhum `user_id` foi versionado, nem como exemplo em documentação.
- **Ainda não conferidas: PUBHEALTH e InSciOut**.

Ver Inventário de datasets e Ampliação do corpus.

RNF11 — Privacidade e LGPD · *em aberto*

O MVP não coleta nem armazena dado pessoal identificável nem histórico de pesquisa. Se RF-A01 e RF-A02 forem admitidos, três condições passam a valer: a senha é guardada com algoritmo de hash com fator de custo (bcrypt ou argon2), o armazenamento depende de consentimento expresso, e o tratamento segue a LGPD.

Por quê: guardar histórico e não guardar dado pessoal são exigências que se chocam, e o projeto precisa declarar qual vale. Consulta sobre saúde é dado sensível pela LGPD, categoria com exigência mais alta que dado pessoal comum.

RNF12 — Só fonte oficial e agência de checagem, com ressalvas declaradas · *medido*

A base é alimentada apenas por agências de checagem reconhecidas e fontes oficiais de saúde. Duas ressalvas acompanham o corpus e precisam estar no portfólio:

- 43% do FactCenter tem o rótulo `boato`, que não gradua nada — a classe "falso" está inflada por construção.
- Há **21 registros verdadeiro em 4.063**. O corpus não serve como fonte de itens verdadeiros para o conjunto de avaliação de RF15, e amostrar isso dele é recusado em código.

RNF13 — Disponibilidade de 90% nas janelas de uso · *em aberto*

O alvo vale para as janelas declaradas de uso, especialmente a sessão de teste com usuário. O protótipo roda localmente e não tem hospedagem contínua, então disponibilidade "do tempo todo" não é uma medida que faça sentido aqui.

RNF14 — Protocolo ético do teste · *especificado*

Toda sessão começa com Termo de Consentimento Livre e Esclarecido e termina com debriefing, em que o participante recebe o gabarito de todos os itens falsos a que foi exposto. Com participantes idosos, o protocolo vai ao comitê de ética antes da coleta.

Por quê: os itens-armadilha de RF15 expõem a pessoa, de propósito, a uma saída errada da IA. Sem debriefing, o teste sai deixando desinformação para trás. Vem da spec `avaliacao-instrumento`.

Pendências antes do Act

Pendência	Requisito	Onde está registrado
Implementar o classificador de fronteira clínica, fora do prompt do gerador	RF07	<code>mvp-copiloto-verificacao</code> , bloco 4, task 4.1
Fechar o catálogo de 6 a 8 técnicas em arquivo versionado	RF10	<code>mvp-copiloto-verificacao</code> , task 3.1

Pendência	Requisito	Onde está registrado
Definir como medir adequação de rótulo, e não só pertinência ao catálogo	RF11	sem task; decisão de grupo pendente
Alinhar a sonda do gerador ao limite de 20 palavras da spec	RNF01	<code>prototipo/teste_gemma4.py:94</code>
Conferir as licenças de PUBHEALTH e InSciOut	RNF10	Estado do projeto
Decidir se conta e histórico entram, e abrir change se entrarem	RF-A01, RF-A02, RNF11	sem change aberto

A padronização do vocabulário de veredito (RF14) e a conferência de integridade textual (RNF07) **já foram executadas**, no change `add-tratamento-datasets-ptbr`.